

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 09 实验名称 网络路由实验

一、 实验内容

1. 基于已有代码框架,实现路由器生成和处理 mOSPF Hello/LSU 消息的相关操作,构建一致性链路状态数据库。
2. 基于实验一,实现路由器计算路由表项的相关操作。

二、 实验流程

2.1 实验一:构建一致性链路状态数据库

我们可以看到在 `mospf_daemon.c` 文件中,对于链路状态使用了四个线程来处理:

1. `sending_mospf_hello_thread`: 定期发送 Hello 消息
2. `sending_mospf_lsu_thread`: 定期发送 LSU 消息
3. `checking_nbr_thread`: 检查邻居状态
4. `checking_database_thread`: 检查链路状态数据库

我们一个一个来实现这些线程的功能。

2.1.1 定期发送 Hello 消息

首先是 `sending_mospf_hello_thread` 线程。该线程的主要功能是定期向所有接口发送 Hello 消息,以便与邻居建立和维护邻居关系。我们需要遍历所有接口,并为每个接口构建并发送 Hello 消息。mOSPF Hello 消息的格式如下:

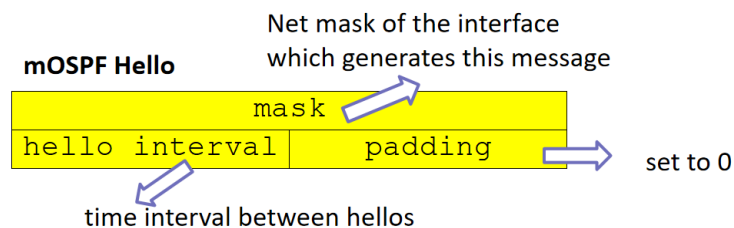


图 1: mOSPF Hello 消息格式

除此之外,我们还需要给 Hello 消息添加 mOSPF 头部。mOSPF 头部的格式如下:

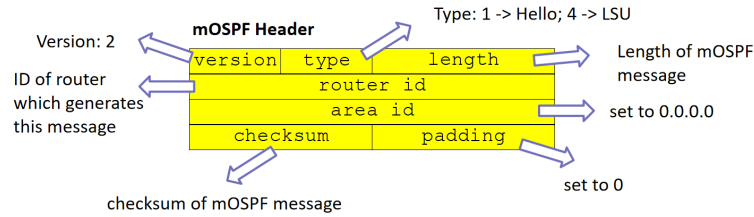


图 2: mOSPF 头部格式

在 mospf_proto.c 文件中,已经实现好了填充 mospf 头部,hello 消息和 lsu 消息的函数,我们只需要调用即可。我们还要添加 ip 头和以太网头。按照约定,mOSPF Hello 消息目的 IP 地址为 224.0.0.5,目的 MAC 地址为 01:00:5E:00:00:05。

Listing 1: 定期发送 Hello 消息代码片段

```
list_for_each_entry(iface, &instance->iface_list, list) {
    int len = MOSPF_HDR_SIZE + MOSPF_HELLO_SIZE + IP_BASE_HDR_SIZE + ETHER_HDR_SIZE;
    char *packet = (char *)malloc(len);
    if (!packet) continue;

    memset(packet, 0, len);

    // 1. Fill MOSPF Header & Body
    struct mospf_hdr *mospf = (struct mospf_hdr *) (packet + ETHER_HDR_SIZE + IP_BASE_HDR_SIZE);
    struct mospf_hello *hello = (struct mospf_hello *) (packet + ETHER_HDR_SIZE + IP_BASE_HDR_SIZE +
        MOSPF_HDR_SIZE);

    mospf_init_hello(hello, iface->mask);
    mospf_init_hdr(mospf, MOSPF_TYPE_HELLO, MOSPF_HDR_SIZE + MOSPF_HELLO_SIZE, instance->router_id,
        instance->area_id);

    mospf->checksum = mospf_checksum(mospf);

    // 2. Fill IP Header
    struct iphdr *ip = (struct iphdr *) (packet + ETHER_HDR_SIZE);
    ip_init_hdr(ip, iface->ip, MOSPF_ALLSPFRouters, MOSPF_HDR_SIZE + MOSPF_HELLO_SIZE +
        IP_BASE_HDR_SIZE, IPPROTO_MOSPF);

    ip->ttl = 1;
    ip->checksum = ip_checksum(ip);

    // 3. Fill Ethernet Header
    u8 multicast_mac[ETH_ALEN] = {0x01, 0x00, 0x5e, 0x00, 0x00, 0x05};
    struct ether_header *eh = (struct ether_header *) packet;
    memcpy(eh->ether_dhost, multicast_mac, ETH_ALEN);
    memcpy(eh->ether_shost, iface->mac, ETH_ALEN);
    eh->ether_type = htons(ETH_P_IP);

    // 4. Send
    iface_send_packet(iface, packet, len);
}
```

```
}
```

2.1.2 定期发送 LSU 消息

接下来是 `sending_mospf_lsu_thread` 线程。该线程的主要功能是定期向所有邻居发送 LSU 消息，以便通告链路状态信息。我们需要遍历链路状态数据库中的所有链路状态条目，并为每个条目构建并发送 LSU 消息。mOSPF LSU 消息的格式如下：

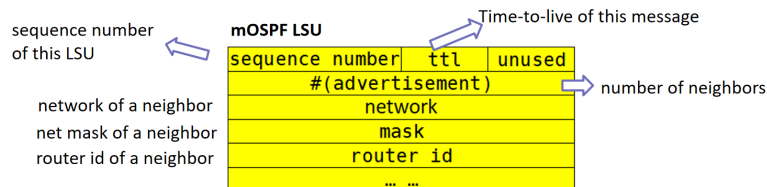


图 3: mOSPF LSU 消息格式

由于目的 mac 在此处不确定,所以使用 `ip_send_packet` 函数发送数据包。其他的和 hello 消息类似。

2.1.3 检查邻居状态

然后是 `checking_nbr_thread` 线程。该线程的主要功能是定期检查邻居状态,并删除超时的邻居。我们需要遍历每个接口的邻居列表,检查每个邻居的状态,如果这个邻居已经超过了邻居超时时间没有收到 Hello 消息,就将其删除。如果没有超过就将其超时时间加 1。

Listing 2: 检查邻居状态代码片段

```
void *checking_nbr_thread(void *param)
{
    // fprintf(stdout, "TODO: neighbor list timeout operation.\n");
    iface_info_t *iface = NULL;
    while (1) {
        pthread_mutex_lock(&mospf_lock);
        list_for_each_entry(iface, &instance->iface_list, list) {
            mospf_nbr_t *nbr = NULL, *nbr_q = NULL;
            list_for_each_entry_safe(nbr, nbr_q, &iface->nbr_list, list) {
                nbr->alive++;
                if (nbr->alive > iface->helloint * 3) {
                    list_delete_entry(&nbr->list);
                    free(nbr);
                    iface->num_nbr--;
                    sending_mospf_lsu();
                }
            }
        }
        pthread_mutex_unlock(&mospf_lock);
        sleep(1);
    }
    return NULL;
}
```

2.1.4 检查链路状态数据库

最后是 `checking_database_thread` 线程。该线程的主要功能是定期检查链路状态数据库，并删除过期的链路状态条目。我们需要遍历链路状态数据库中的所有链路状态条目，检查每个条目的状态，如果这个条目已经超过了链路状态超时时间没有收到 LSU 消息，就将其删除。这个实现和检查邻居状态类似。

除了上述四个线程函数之外，我们还需要实现两个函数分别用于处理到的 mOSPF Hello 消息和 LSU 消息，分别是 `handle_mospf_hello` 和 `handle_mospf_lsu` 函数。

2.1.5 处理 mOSPF 消息

对于生成一致链路状态数据库来说，最重要的就是处理收到的 mOSPF 消息了。首先是 `handle_mospf_hello` 函数。该函数的主要功能是处理收到的 mOSPF Hello 消息，并更新邻居列表。

对于接受到的 Hello 消息，首先我们需要提取出消息中的 `rid`, `src_ip` 和 `mask` 字段，检查 `rid` 是否存在于邻居列表中。如果存在，就将其超时时间清零；如果不存在，就创建一个新的邻居条目，并将其添加到邻居列表中。并且发送 LSU 消息。

Listing 3: 处理 mOSPF Hello 消息代码片段

```
void handle_mospf_hello(iface_info_t *iface, const char *packet, int len)
{
    // fprintf(stdout, "TODO: handle mOSPF Hello message.\n");
    struct mospf_header *eh = (struct mospf_header *)packet;
    struct iphdr *ip = (struct iphdr *) (packet + ETHER_HDR_SIZE);
    struct mospf_hdr *mospf = (struct mospf_hdr *) ((char *)ip + IP_HDR_SIZE(ip));
    struct mospf_hello *hello = (struct mospf_hello *) ((char *)mospf + MOSPF_HDR_SIZE);

    u32 rid = ntohl(mospf->rid);
    u32 src_ip = ntohl(ip->saddr);
    u32 mask = ntohl(hello->mask);

    pthread_mutex_lock(&mospf_lock);

    int found = 0;
    mospf_nbr_t *nbr = NULL;

    list_for_each_entry(nbr, &iface->nbr_list, list) {
        if (nbr->nbr_id == rid) {
            found = 1;
            nbr->alive = 0;
            break;
        }
    }

    if (!found) {
        nbr = (mospf_nbr_t *) malloc(sizeof(mospf_nbr_t));
        if (nbr) {
            nbr->nbr_id = rid;
            nbr->nbr_ip = src_ip;
            nbr->nbr_mask = mask;
            nbr->alive = 0;
            list_add_tail(&nbr->list, &iface->nbr_list);
        }
    }
}
```

```

        iface->num_nbr++;
        sending_mospf_lsu();
    }
}
pthread_mutex_unlock(&mospf_lock);
}

```

对于接受到的 LSU 消息,我们需要提取出消息中的 rid,seq,ttl 和 nadv 字段,检查 rid 是否存在于链路状态数据库中。如果存在,且该条目的序列号小于收到的序列号,就更新该条目的信息;如果不存在,就创建一个新的链路状态条目,并将其添加到链路状态数据库中。然后更新路由表(这是后续动态路由表计算的部分)。最后,我们还需要将该 LSU 的 ttl 减 1,如果 ttl 大于 0,就将该 LSU 转发给所有邻居。这一段代码比较长,而且也 and hello 的消息处理比较的相像。

2.2 实验二:计算路由表项

在完成实验一后,我们已经构建好了链路状态数据库,在启动网络一端时间后,每个节点的链路状态就会收敛到一个稳定状态。此时每个节点都拥有了一份网络的完整拓扑信息了,接下来我们就可以使用 Dijkstra 算法来计算最短路径,从而生成路由表项了。笔者单独实现了一个 update_rtable 函数用于更新路由表。

具体来说就是按照 Dijkstra 算法的步骤来实现的。首先定义数据结构:

Listing 4: Dijkstra 算法数据结构代码片段

```

typedef struct {
    u32 rid;           // Router ID
    u32 dist;          // 距离
    int visited;       // 是否已访问
    u32 next_hop;      // 下一跳 IP (对于第一跳)
    iface_info_t *iface; // 出接口 (对于第一跳)
} dijkstra_node_t;

```

此处定义了一个 dijkstra_node_t 结构体用于存储 Dijkstra 算法中的节点信息,包括路由器 ID、当前最短距离、是否已访问、以及到达该节点所需的下一跳 IP 和出接口。

在网络路由动态计算的实现中,与标准 Dijkstra 算法仅计算最短路径树不同,我们需要在松弛(Relaxation)过程中传递转发信息。即:对于源节点的直连邻居,记录实际的下一跳和出接口;对于更远的节点,其下一跳和出接口直接继承自其前驱节点。

算法流程为:按照距离从小到大依次选取未访问节点 u。若 u 为源节点,则初始化其邻居的转发信息;若 u 为远端节点,则遍历其 LSA 中的网络信息,若该网络对应的路由未被计算过,则使用节点 u 中保存的下一跳和出接口信息生成新的路由表项;同时对 u 的邻居进行松弛操作,直到所有节点均被访问过为止。

相关代码比较长,这里不再赘述,完整代码见源文件。

三、实验结果

实验所用的网络拓扑如图所示:

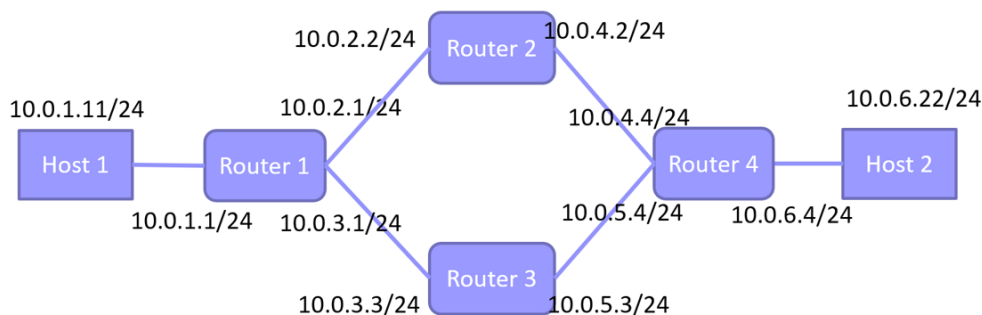


图 4: 实验网络拓扑

3.1 实验一结果

我们实现了一个函数用于打印链路状态数据库:

Listing 5: 打印链路状态数据库代码片段

```
void print_mospf_db()
{
    pthread_mutex_lock(&mospf_lock);

    fprintf(stdout, "MOSPF Database entries:\n");
    fprintf(stdout, "RID\t\tNetwork\t\tMask\t\tNeighbor\n");
    fprintf(stdout, "-----\n");

    mospf_db_entry_t *db_entry = NULL;
    list_for_each_entry(db_entry, &mospf_db, list) {
        for (int i = 0; i < db_entry->nadv; i++) {
            struct mospf_lsa *lsa = &db_entry->array[i];
            u32 db_rid = db_entry->rid;
            u32 net = ntohl(lsa->network);
            u32 m = ntohl(lsa->mask);
            u32 neigh = ntohl(lsa->rid);
            fprintf(stdout, IP_FMT "\t" IP_FMT "\t" IP_FMT "\t" IP_FMT "\n",
                    HOST_IP_FMT_STR(db_rid),
                    HOST_IP_FMT_STR(net),
                    HOST_IP_FMT_STR(m),
                    HOST_IP_FMT_STR(neigh));
        }
    }
    fprintf(stdout, "\n");

    pthread_mutex_unlock(&mospf_lock);
}
```

在启动 mininet 网络后,将输出重定向到 log 文件中,然后等待一段时间让链路状态收敛,最后查看 log 文件中的链路状态数据库内容,如图所示:

MOSPF Database entries:			
RID	Network	Mask	Neighbor

10.0.2.2	10.0.2.0	255.255.255.0	10.0.1.1
10.0.2.2	10.0.4.0	255.255.255.0	10.0.4.4
10.0.3.3	10.0.3.0	255.255.255.0	10.0.1.1
10.0.3.3	10.0.5.0	255.255.255.0	10.0.4.4
10.0.4.4	10.0.4.0	255.255.255.0	10.0.2.2
10.0.4.4	10.0.5.0	255.255.255.0	10.0.3.3
10.0.4.4	10.0.6.0	255.255.255.0	0.0.0.0

(a) r1 节点链路状态数据库内容

MOSPF Database entries:			
RID	Network	Mask	Neighbor

10.0.4.4	10.0.4.0	255.255.255.0	10.0.2.2
10.0.4.4	10.0.5.0	255.255.255.0	10.0.3.3
10.0.4.4	10.0.6.0	255.255.255.0	0.0.0.0
10.0.3.3	10.0.3.0	255.255.255.0	10.0.1.1
10.0.3.3	10.0.5.0	255.255.255.0	10.0.4.4
10.0.1.1	10.0.1.0	255.255.255.0	0.0.0.0
10.0.1.1	10.0.2.0	255.255.255.0	10.0.2.2
10.0.1.1	10.0.3.0	255.255.255.0	10.0.3.3

(b) r2 节点链路状态数据库内容

MOSPF Database entries:			
RID	Network	Mask	Neighbor

10.0.1.1	10.0.1.0	255.255.255.0	0.0.0.0
10.0.1.1	10.0.2.0	255.255.255.0	10.0.2.2
10.0.1.1	10.0.3.0	255.255.255.0	10.0.3.3
10.0.4.4	10.0.4.0	255.255.255.0	10.0.2.2
10.0.4.4	10.0.5.0	255.255.255.0	10.0.3.3
10.0.4.4	10.0.6.0	255.255.255.0	0.0.0.0
10.0.2.2	10.0.2.0	255.255.255.0	10.0.1.1
10.0.2.2	10.0.4.0	255.255.255.0	10.0.4.4

(c) r3 节点链路状态数据库内容

MOSPF Database entries:			
RID	Network	Mask	Neighbor

10.0.2.2	10.0.2.0	255.255.255.0	10.0.1.1
10.0.2.2	10.0.4.0	255.255.255.0	10.0.4.4
10.0.3.3	10.0.3.0	255.255.255.0	10.0.1.1
10.0.3.3	10.0.5.0	255.255.255.0	10.0.4.4
10.0.1.1	10.0.1.0	255.255.255.0	0.0.0.0
10.0.1.1	10.0.2.0	255.255.255.0	10.0.2.2
10.0.1.1	10.0.3.0	255.255.255.0	10.0.3.3

(d) r4 节点链路状态数据库内容

图 5: 四节点链路状态数据库

可以看到,四个路由器节点的链路状态数据库内容均正确,且一致,说明链路状态已经收敛。

3.2 实验二结果

路由表的打印可以直接调用函数 `print_rtable` 函数。活得输出的方法和上面类似。等待一段时间让路由表收敛后,查看 `log` 文件中的路由表内容,如图所示:

Routing Table:			
dest	mask	gateway	if_name

10.0.1.0	255.255.255.0	0.0.0.0	r1-eth0
10.0.2.0	255.255.255.0	0.0.0.0	r1-eth1
10.0.3.0	255.255.255.0	0.0.0.0	r1-eth2
10.0.4.0	255.255.255.0	10.0.2.2	r1-eth1
10.0.5.0	255.255.255.0	10.0.3.3	r1-eth2
10.0.6.0	255.255.255.0	10.0.2.2	r1-eth1

(a) r1 节点路由表内容

Routing Table:			
dest	mask	gateway	if_name

10.0.2.0	255.255.255.0	0.0.0.0	r2-eth0
10.0.4.0	255.255.255.0	0.0.0.0	r2-eth1
10.0.1.0	255.255.255.0	10.0.2.1	r2-eth0
10.0.3.0	255.255.255.0	10.0.2.1	r2-eth0
10.0.5.0	255.255.255.0	10.0.4.4	r2-eth1
10.0.6.0	255.255.255.0	10.0.4.4	r2-eth1

(b) r2 节点路由表内容

Routing Table:			
dest	mask	gateway	if_name

10.0.3.0	255.255.255.0	0.0.0.0	r3-eth0
10.0.5.0	255.255.255.0	0.0.0.0	r3-eth1
10.0.1.0	255.255.255.0	10.0.3.1	r3-eth0
10.0.2.0	255.255.255.0	10.0.3.1	r3-eth0
10.0.4.0	255.255.255.0	10.0.5.4	r3-eth1
10.0.6.0	255.255.255.0	10.0.5.4	r3-eth1

(c) r3 节点路由表内容

Routing Table:			
dest	mask	gateway	if_name

10.0.4.0	255.255.255.0	0.0.0.0	r4-eth0
10.0.5.0	255.255.255.0	0.0.0.0	r4-eth1
10.0.6.0	255.255.255.0	0.0.0.0	r4-eth2
10.0.2.0	255.255.255.0	10.0.4.2	r4-eth0
10.0.3.0	255.255.255.0	10.0.5.3	r4-eth1
10.0.1.0	255.255.255.0	10.0.4.2	r4-eth0

(d) r4 节点路由表内容

图 6: 四节点路由表

可以看到,四个路由器节点的路由表内容均正确,且一致,说明路由表已经收敛。

第二个实验小项,使用 traceroute 测试 h1 到 h2 的路径,随后断开 r2 和 r4 节点的连接,经过一段时间后再使用 traceroute 测试 h1 到 h2 的路径,结果如下所示:

```
kouyixin@ubuntu:~/CNLab/09-mospf/code$ sudo python3 topo.py
mininet> h1 traceroute h2
traceroute to 10.0.6.22 (10.0.6.22), 30 hops max, 60 byte packets
 1  10.0.1.1 (10.0.1.1)  0.211 ms  0.154 ms  0.149 ms
 2  10.0.2.2 (10.0.2.2)  0.292 ms  0.288 ms  0.285 ms
 3  10.0.4.4 (10.0.4.4)  0.345 ms  0.344 ms  0.341 ms
 4  10.0.6.22 (10.0.6.22)  0.304 ms  0.311 ms  0.312 ms
mininet>
mininet> link r2 r4 down
mininet> h1 traceroute h2
traceroute to 10.0.6.22 (10.0.6.22), 30 hops max, 60 byte packets
 1  10.0.1.1 (10.0.1.1)  0.200 ms  0.047 ms  0.041 ms
 2  10.0.3.3 (10.0.3.3)  0.491 ms  0.519 ms  0.516 ms
 3  10.0.5.4 (10.0.5.4)  2.356 ms  2.698 ms  2.703 ms
 4  10.0.6.22 (10.0.6.22)  2.701 ms  2.701 ms  2.699 ms
```

图 7: traceroute 测试结果

可以看到,第一次 traceroute 时,数据包经过 r1、r2、r4 到达 h2;断开 r2 和 r4 节点的连接后,经过一段时间路由表收敛,再次 traceroute 时,数据包经过 r1、r3、r4 到达 h2。

验证了路由器能够根据链路状态动态计算路由表,并在网络拓扑变化时及时更新路由表,保证数据包能够正确到达目的地。

四、 实验中遇到的问题

在给 mOSPF LSU 消息转发时,最开始笔者错误地使用了 iface_send_packet 函数发送数据包,导致数据包无法正确发送到目的地。但是因为 mOSPF LSU 消息的目的 MAC 地址是不确定的,所以应该使用 ip_send_packet 函数发送数据包。修改后问题解决。

除此之外,在给 mOSPF LSU 消息添加以太头的时候,一开始考虑拷贝 0 到目的 MAC 地址,但是由于 memcpy 函数的参数置 0 会导致拷贝 0 地址的信息,最终使得程序崩溃。后来改为用 memset 函数将目的 MAC 地址置 0,问题解决。